COLLEGE CODE: 8203

COLLEGE: AVC COLLEGE OF ENGINEERING

DEPARTMENT: INFORMATION TECHNOLOGY

STUDENT NM-ID: 30AFAE4B795F72FB39C41F86182C47A1

ROLL NO: 23IT112

DATE:22/09/2025

Completed the project named as

Phase_3 TECHNOLOGY PROJECT

NAME: Online Quiz Application

SUBMITTED BY,

NAME: VALLI C

MOBILE NO: 6374458528

MVP IMPLEMENTATION:

PROJECT SETUP:

The Online Quiz Application starts with setting up the backend using ***Node.js and Express***. ***MongoDB*** will be configured to store quiz questions, options, and user data. A proper folder structure is created to separate routes, models, and controllers.

The frontend is initialized using ***React*** to display quizzes and results dynamically***. REST APIs*** are connected to fetch questions and submit answers. Environment files are added to manage database and API configuration securely.

For collaboration***, GitHub*** is used for version control and tracking changes. The repository helps in managing project updates and team contributions. Initial testing ensures APIs and database connections are working correctly.

CORE FEATURE IMPLEMENTATION:

The MVP of the Online Quiz Application focuses on implementing the essential features that ensure smooth quiz creation, participation, and evaluation.

Quiz Management (Backend – MongoDB + APIs):

- Store multiple-choice quiz questions and options in MongoDB.
- Provide REST APIs to fetch quizzes dynamically and to submit answers.
- Admin APIs enable adding, updating, and deleting quiz questions.

Quiz Participation (Frontend – React):

- React renders questions dynamically by fetching them from the backend.
- Local state temporarily stores user-selected answers during the quiz.
- On submission, answers are sent to the backend for evaluation.

Answer Evaluation & Scoring:

- Backend checks submitted answers against stored correct answers.
- Calculates total score and sends it back to the frontend.
- Frontend displays results instantly after submission.

User History

- Store user responses, scores, and quiz attempts in MongoDB for future tracking.
- Enables personalized progress tracking and insights.

Scalability & Flexibility:

- Modular APIs ensure easy integration of new quiz categories and difficulty levels.
- Frontend design allows for future extension to timed quizzes, leaderboards, or authentication.

DATA STORAGE:

Data storage will be handled using a combination of **local state management** (for temporary data in the frontend) and **MongoDB** database (for persistent storage in the backend).

Frontend (Local State):

React manages quiz flow using local component state.

Selected answers are temporarily stored in local state until the quiz is submitted.

Local state ensures smooth navigation between questions without frequent server requests.

Quiz results are displayed immediately using the locally stored responses and the backend response.

Backend (Database – MongoDB):

Quiz questions, options, and correct answers are stored in MongoDB collections.

User responses and scores can optionally be stored for maintaining quiz history.

Admin APIs update the database when new questions are added or existing ones are modified.

Schema design (example):

Questions Collection: { questionText, options, correctAnswer, category, difficulty }

UserResponses Collection (optional): { userId, quizId, answers, score, timestamp }

Flow of Data Storage:

Admin inserts/updates quiz questions in MongoDB.

Frontend fetches quiz data via REST APIs.

User answers are maintained in React local state until submission.

On submission, answers are sent to the backend for validation, scoring, and optional storage in MongoDB.

TESTING CORE FEATURES:

To ensure the Online Quiz Application MVP works smoothly, core features will be tested through functional and integration testing.

Quiz Data Retrieval:

- Verify that quizzes are correctly fetched from **MongoDB** via **REST APIs**.
- Test for random quiz selection and category-based quiz retrieval (if enabled).

Answer Submission & Scoring:

- Check that user answers are submitted successfully to the **backend.**
- Validate that scoring logic correctly compares answers with the stored correct answers.
- Test edge cases like unanswered questions or invalid submissions.

Admin APIs:

- Test quiz creation, update, and deletion endpoints.
- Ensure data consistency **in MongoDB** after admin operations.

Frontend Quiz Flow (React):

- Confirm that dynamic questions render properly.
- Test navigation between questions and persistence of selected answers in local state.
- Verify that results display correctly after submission.

Integration Testing:

- Validate end-to-end flow**: quiz fetch → answer selection → submission → scoring → result display.**
- Ensure smooth data flow between frontend, backend, and database.

Performance & Reliability:

- Test application with multiple users taking quizzes simultaneously.
- Verify **API response** times and error handling for failed requests.

VERSION CONTROL:

Maintain project integrity and allow for collaboration

- **Repository:** https://github.com/valli-chandrasekar/NM-AVC-IT-VALLI
- **Commits:** Regularly commit changes with descriptive messages after completing each major feature or bug fix.